

Roteiro para os testes de Selenium

1. Escolher o fluxo de negócio

No Figma, identificar a tela inicial, a ação principal e o que deve aparecer depois. Escrever dois resultados esperados: um para o uso correto e outro para uma entrada inválida ou incompleta.

2. Iniciar o projeto React

Criar um app react vazio, instalá-lo e confirmar que abre no navegador.

3. Montar a estrutura em React

Traduzir a tela escolhida para componentes React: títulos, campos, botões, lista ou cartão de resultado. Usar texto e dados de exemplo.

4. Aplicar o visual do Figma

Ajustar cores, espaçamento, tipografia e imagens exportadas. Priorizar legibilidade e semelhança geral, sem tentar aperfeiçoar cada detalhe.

5. Fazer a interação funcionar

Usar o estado (state) do React para responder ao clique, atualizar a tela e mostrar o resultado. Implementar também a validação escolhida.

6. Preparar a tela para testes

Garantir que campos tenham rótulos e IDs estáveis; que botões sejam botões HTML; e que mensagens de sucesso ou erro possam ser localizadas.

7. Escrever e rodar os testes

Automatizar o fluxo principal e a validação no navegador. Corrigir falhas de localização (posição) ou comportamento.

8. Entregar e demonstrar

Confirmar que outro integrante consegue iniciar o app e rodar o teste. Fazer uma demonstração breve do fluxo.

Passo a Passo Inicial

1. Criar o projeto em react (aqui estou usando o vite)

```
npm create vite@latest app-alunos -- --template react
cd app-alunos
npm install
npm run dev
```

O Vite mostrará o endereço local; normalmente é <http://localhost:5173>.

2. Edite o arquivo App.jsx, tem um exemplo neste repositório:

https://github.com/mikltpcs3643_Risco/blob/main/app-alunos/src/App.jsx

```
return (
  <main className="pagina">
    <section className="cartao" aria-labelledby="titulo-pagina">
      <p className="sobretitulo">Protótipo convertido em React</p>
      <h1 id="titulo-pagina">Registrar item</h1>
      <p>Preencha o título e salve para ver o resultado.</p>

      <form onSubmit={handleSubmit} noValidate>
        <label htmlFor="titulo">Título</label>

        <input
          id="titulo"
          name="titulo"
          type="text"
          value={titulo}
          onChange={handleChange}
          aria-invalid={estado === "erro"}
          aria-describedby={estado === "erro" ? "feedback" : undefined}
        />

        <button type="submit" data-testid="acao-principal">
          Salvar
        </button>
      </form>

      {mensagem && (
        <p
          id="feedback"
          className={`feedback feedback--${estado}`}
          role={estado === "erro" ? "alert" : "status"}
        >
          {mensagem}
        </p>
      )}
    </section>
  </main>
);
```

Depois, edite também o .css

https://github.com/miklt/pcs3643_Risco/blob/main/app-alunos/src/App.css

Os seletores ficam prontos para os testes:

- campo: `By.ID, "titulo"`
- botão: `[data-testid="acao-principal"]`
- mensagem de sucesso: `[role="status"]`
- mensagem de erro: `[role="alert"]`

Por exemplo, no teste de sucesso, digite **Teste** e verifique que a mensagem exibida é **Salvo: Teste**. No teste de validação, deixe o campo vazio e verifique que aparece **0 título é obrigatório**.

3. Criar e executar os testes Selenium em Python

O exemplo abaixo testa a interface React do item 2: um teste verifica o salvamento e outro a mensagem de erro para o campo vazio. O código usa esperas explícitas para aguardar as mensagens aparecerem, em vez de pausas fixas.

O Selenium Manager vem incluído no Selenium e pode gerenciar o driver automaticamente quando ele não está disponível. Na primeira execução, pode ser necessário acesso à internet para obter o driver do Chrome.

O código fonte do teste em selenium está disponível em:

https://github.com/miklt/pcs3643_Risco/tree/main/selenium

Um exemplo é:

```
def test_salva_titulo_e_exibe_confirmacao(self):  
    campo_titulo = self.driver.find_element(By.ID, "titulo")  
    campo_titulo.send_keys("Teste Selenium")  
    botao_salvar = self.driver.find_element(  
        By.CSS_SELECTOR,  
        '[data-testid="acao-principal"]'  
    )
```

```
botao_salvar.click()

mensagem = self.wait.until(
    EC.visibility_of_element_located(
        (By.CSS_SELECTOR, '[role="status"]')
    )
)

self.assertEqual(mensagem.text, "Salvo: Teste Selenium")

self.assertEqual(campo_titulo.get_attribute("value"), "")
```